

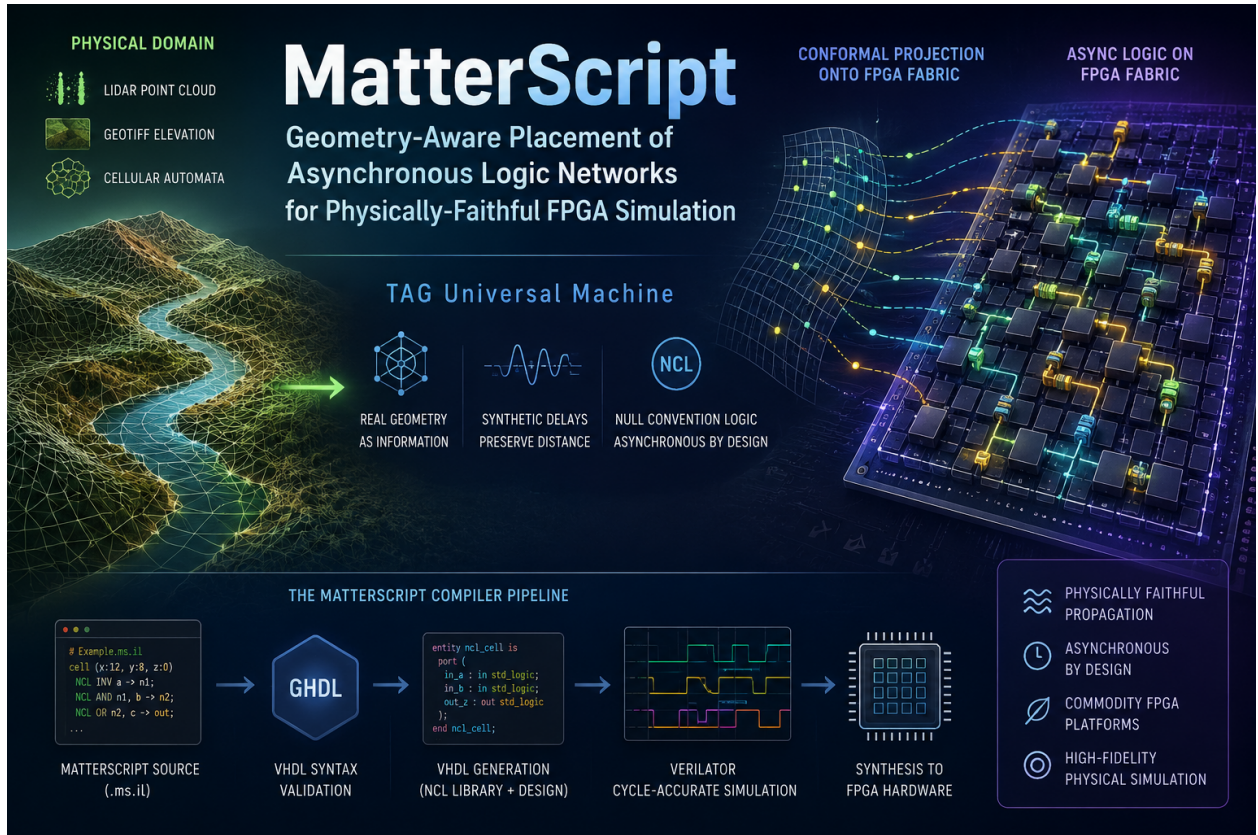
MatterScript: Geometry-Aware Placement of Asynchronous Logic Networks for Physically-Faithful FPGA Simulation

TAG Universal Machine

July 18, 2026

Abstract

We present MatterScript, a domain-specific language and compiler toolchain for geometry-aware placement of asynchronous logic networks on commodity FPGA hardware. MatterScript introduces the principle that the physical domain being simulated should determine the spatial arrangement of logic elements on the FPGA fabric, with synthetic propagation delays encoding geometric distances that cannot be preserved exactly by the projection. This produces simulations whose propagation behavior mirrors the physical domain without requiring custom hardware substrates. The system is grounded in Karl Fant’s Null Convention Logic and Invocation Language, which make propagation delay a first-class computational parameter rather than a side effect to be minimized. The compiler pipeline proceeds from `.ms.il` source through GHDL syntax validation to Verilator cycle-accurate simulation, and is designed to extend to standard FPGA synthesis tools. MatterScript additionally provides cellular automata as generative primitives for synthetic domain geometry, and finite state machine descriptions as a convenience layer over the full Invocation Language model. Beyond its technical contributions, MatterScript embodies a methodology for collaborative world modeling: building physically-faithful executable models of real-world systems one domain at a time, in partnership between human engineers and AI systems, with hardware locking each verified layer into a form that neither needs to re-derive.



MatterScript compiler pipeline — from domain geometry through NCL logic synthesis to FPGA simulation

Contents

1	Introduction	4
2	Background	4
2.1	The Limitations of Clock-Driven FPGA Simulation	4
2.2	Null Convention Logic	5
2.3	Karl Fant’s Invocation Language	5
2.4	Integer Quantization	6
2.5	Conformal Mapping	6
3	The MatterScript Language	6
3.1	Design Philosophy	6
3.2	The FSM Dialect	6
3.3	The Invocation Language Dialect	7
3.3.1	Signal Encoding	7
3.3.2	Name Composition and ROM Synthesis	7
3.4	The Table Generator	8

3.5	The Component Layer Convention	8
3.6	The Geo Dialect	9
4	Geometry-Aware Placement	9
4.1	The Two-Phase Pipeline	9
4.2	Conformal Projection and Delay Synthesis	9
4.3	Cell Reference Syntax	10
4.4	Comparison with Conventional FPGA Simulation	10
5	Collaborative World Modeling: AI, Engineers, and Executable Physics	10
5.1	The World Model Problem	10
5.2	The Radiator Example: A Case Study in Collaborative Modeling	11
5.3	Lateral Scaling: From Radiator to Drivetrain	12
5.4	The Role of AI in This Process	13
6	The Compiler Pipeline	13
6.1	Overview	13
6.2	The IL Parser	14
6.3	VHDL Generation	14
6.4	Build System	14
7	Implementation Status	15
8	Related Work	15
9	Conclusion	15

1 Introduction

Conventional FPGA-based simulation treats the logic fabric as a generic computing substrate. Place-and-route tools optimize for minimum wire length and maximum clock frequency, packing logic as densely as possible without regard for any geometric meaning the circuit might carry. A fluid dynamics simulation of a riverbed occupies the same undifferentiated grid as any other computation of similar size.

This paper argues that for physical simulation workloads, this is the wrong optimization objective. When simulating a physical domain — a fluid system, a structural load, an electromagnetic field — the geometry of the domain *is* information. Cells that are adjacent in the physical domain should be adjacent on the logic fabric. Cells that are far apart in the physical domain should have proportionally longer signal paths between them. The propagation of information through the simulation should mirror the propagation of effects through the physical system.

MatterScript makes this concrete through three connected ideas:

1. **Geometry-aware placement:** physical domain geometry, sourced from LIDAR point clouds, GeoTIFF elevation data, parametric CAD models, or cellular automata generation, is projected onto the FPGA fabric using conformal mapping. Logic cells are placed at projected positions.
2. **Delay synthesis:** distances distorted by the projection are compensated with synthetic delay elements, so that propagation timing across the fabric matches geodesic distances in the original domain.
3. **Null Convention Logic:** Karl Fant’s asynchronous logic model [1] makes propagation delay a computational parameter rather than a clock-constrained nuisance. NCL circuits fire when their inputs are complete, not when a clock edge arrives, making delay synthesis a natural and meaningful operation.

The result is a simulation that runs on a cheap commodity FPGA but whose propagation behavior is faithful to the physical geometry of the domain being modeled — analogous to a wind tunnel, but implemented in reconfigurable digital logic. Beyond the technical contributions, this paper describes a methodology for collaborative world modeling in which human engineers and AI systems work together to make physical knowledge explicit and executable, domain by domain, with hardware verifying each layer before the next is built.

2 Background

2.1 The Limitations of Clock-Driven FPGA Simulation

Synchronous FPGA designs coordinate computation with a global clock signal. Every register updates on the same clock edge, regardless of whether the data feeding it has meaningfully changed. For physical simulations this creates several problems:

- **Artificial synchronization:** physical phenomena do not propagate synchronously. A pressure wave in a fluid does not update every cell simultaneously; it travels at a finite speed determined by the medium. Clocked simulation imposes a uniform time step that obscures this.
- **Geometry-blind placement:** standard place-and-route has no mechanism to express that two cells should be placed far apart because they represent physically distant locations.
- **Staircase boundary approximation:** mapping irregular physical domains onto a regular grid introduces staircase artifacts at boundaries that affect simulation accuracy.

2.2 Null Convention Logic

Null Convention Logic [1] is an asynchronous logic paradigm that eliminates the global clock. Every signal carries a *completeness* indicator in addition to its data value. A signal in the NULL state carries no data. A signal in the DATA state carries a valid value. Computation proceeds when all inputs to a function transition from NULL to DATA.

For geometry-aware simulation, NCL has a decisive advantage: propagation delay is not noise to be minimized but a property of the network that can be deliberately designed. A synthetic delay element between two logic cells is not a timing violation — it is a faithful representation of the physical distance between the locations those cells represent.

Additional properties relevant to physical simulation:

- **Natural pipeline correctness:** a cell cannot update until all neighbor inputs are complete, preventing data hazards without explicit pipeline staging.
- **Network transparency:** the completeness handshake operates identically across on-chip connections, board-level buses, and network links, enabling seamless distribution of large simulations.
- **Sequentiality as emergence:** sequential behavior arises from data dependency chains rather than imposed clock ordering, correctly modeling the causal structure of physical propagation.

2.3 Karl Fant’s Invocation Language

Fant’s Invocation Language (IL) [1] provides a formal notation for NCL computation. The fundamental concept is the *theng*: something that has a location in the network and asserts a value. Destination places (written **\$name**) receive tokens from the outer context. Source places (written **name<>**) emit tokens once complete.

The IL’s most distinctive feature is *name composition*: destination place values are concatenated to form lookup keys into constant tables, implementing arbitrary combinational logic without conditional syntax. A composition **\$a\$b()** with **\$a = 1** and **\$b = 3** forms the key 13 and returns the corresponding table value. If no entry matches, the place remains NULL indefinitely.

2.4 Integer Quantization

MatterScript adopts a strict integer-only computation model. All token values are non-negative integers in the range $[0, 127]$, encoded in 7 bits. No floating point arithmetic appears anywhere in the language, compiler, or generated hardware.

This is a principled design choice, not a simplification. Floating point operations require breaking a token into mantissa and exponent fields, routing them to a shared FPU, performing the operation, and reassembling — a pipeline that reintroduces the CPU-like data movement that NCL is designed to eliminate. Integer quantization keeps tokens atomic: each token travels whole and indivisible through the network, and all computation is local. Quantization resolution is a design parameter controlled by the choice of scale convention in the physics expressions.

2.5 Conformal Mapping

A conformal map preserves angles locally while allowing area distortion. For projecting a physical domain onto an FPGA fabric, conformal mapping minimizes distortion of local geometry while fitting the domain into the available rectangular grid. MeshLab [3] implements several conformal UV parameterization algorithms suitable for this purpose.

3 The MatterScript Language

3.1 Design Philosophy

MatterScript is organized around three dialects sharing a common flat, keyword-driven syntax: **.ms.fsm** for finite state machines, **.ms.il** for Invocation Language networks, and **.ms.geo** for cellular automata substrate descriptions. A fourth dialect, **.ms.ca**, is under development for coupled physical simulation with geometry-aware placement.

3.2 The FSM Dialect

Listing 1: CoffeeShop FSM

```
machine CoffeeShop

state Greeting
state TakeOrder
state Payment
state PrepareDrink
state Complete

event CustomerArrived
event OrderPlaced
event PaymentApproved
event DrinkReady
```

```

transition Greeting TakeOrder CustomerArrived
transition TakeOrder Payment OrderPlaced
transition Payment PrepareDrink PaymentApproved
transition PrepareDrink Complete DrinkReady

```

The FSM dialect compiles to two-process VHDL. It is a convenience layer: in the full MatterScript model, finite state machines are IL networks in which exactly one token is in flight at any time — sequentiality as a degenerate case of concurrency.

3.3 The Invocation Language Dialect

Listing 2: 2-bit integer adder in IL

```

add[($a $b)(result<>)
  result<$a$b(>)
|
  $a$b() : 00:0 01:1 02:2 03:3
           10:1 11:2 12:3 13:4
           20:2 21:3 22:4 23:5
           30:3 31:4 32:5 33:6
]

add((1 3)(x<>))

```

3.3.1 Signal Encoding

`signal[7:0] = {data[6:0], valid}`

`signal[0]` is the valid (completeness) bit. `signal[7:1]` carries the 7-bit data payload, supporting up to 127 distinct token values. This encoding is uniform across all place kinds so connections between networks require no type conversion.

3.3.2 Name Composition and ROM Synthesis

Listing 3: Generated VHDL for the add definition (excerpt)

```

ab_key <= a(7 downto 1) & b(7 downto 1);

process(ab_key, complete) begin
  ab_data <= (others => '0');
  ab_valid <= '0';
  if complete = '1' then
    case ab_key is
      when "00000010000011" => ab_data <= "100"; ab_valid <= '1';
      when others => null;
    end case;
  end if;
end process;

```

```

    end if;
end process;

result <= ab_data & ab_valid;

```

3.4 The Table Generator

For simulation cells with large neighborhoods, writing lookup tables by hand is impractical. A heat conduction cell with four 3-bit neighbor inputs has $8^4 = 4096$ table entries. MatterScript provides a **generate** block that computes the table from a physics expression at compile time:

Listing 4: Table generator syntax

```

HeatCell[($n $s $e $w)(temp_out<>)
  temp_out<$n$s$e$w(>
|
  $n$s$e$w() : generate {
    clamp(($n + $s + $e + $w) / 4, 0, 7)
    inputs: $n 0..7, $s 0..7, $e 0..7, $w 0..7
    output: 0..7
  }
]

```

The compiler enumerates all input combinations, evaluates the expression in integer arithmetic, clamps the result to the output range, and emits the VHDL case statement. No floating point arithmetic appears at any stage. The expression language supports addition, subtraction, multiplication, integer division, **clamp**, **avg**, and named integer constants.

Cellular automata are a degenerate case of this mechanism: a CA rule is a **generate** block with a small neighborhood and a Boolean expression. The table generator generalizes this to arbitrary neighborhood sizes and integer-valued physics expressions, applying the same principle uniformly across problem domains.

3.5 The Component Layer Convention

Simulation cells in the standard library follow a port convention that standardizes connectivity across problem domains:

- **Layer 0 (destinations)**: neighborhood input tokens
- **Layer 1 (resolution area)**: local physics computation
- **Layer 2 (sources)**: neighborhood output tokens
- **Layer 3 (control, optional)**: sensor and actuator cell[] reference ports

This convention allows the placement engine to connect any conforming cell definition to any neighbor without knowing its internals — analogous to standardized IC pinouts. It

also enables composable physics: a fluid layer and a thermal layer on the same mesh couple through shared boundary-layer tokens without either knowing the other’s implementation.

3.6 The Geo Dialect

Listing 5: Synthetic terrain via Rule 30

```
namespace terrain
seed entropy42
cald rule 30 width 128 steps 64
height_scale 2.4
export obj terrain.obj
```

The CA spacetime diagram is interpreted as a height map, producing a triangulated mesh exported as OBJ or STL. For real-world applications, the geo dialect is bypassed and meshes are supplied from CAD export, LIDAR, or GeoTIFF sources via MeshLab preprocessing.

4 Geometry-Aware Placement

4.1 The Two-Phase Pipeline

Phase 1: Domain geometry acquisition and preprocessing. The physical domain geometry is acquired from measurement or parametric CAD. MeshLab converts the source data to a triangulated mesh, computes geodesic distances, performs UV parameterization, and annotates vertices with curvature and local area distortion.

Phase 2: Logic synthesis and geometry-aware placement. MatterScript reads the annotated mesh and maps the IL network onto the FPGA fabric. Each theng is assigned to a mesh vertex. The UV parameterization provides 2D coordinates scaled to the FPGA fabric dimensions. Synthetic delay elements are inserted proportional to the distance distortion introduced by the projection.

4.2 Conformal Projection and Delay Synthesis

Area distortion introduced by the conformal map is computed per-edge as:

$$\delta_{ij} = \left| \frac{d_{UV}(i, j)}{d_{geo}(i, j)} - 1 \right|$$

A synthetic delay proportional to δ_{ij} is inserted on the connection between thengs i and j . In NCL, delay elements of varying lengths can be freely mixed without affecting timing closure:

Listing 6: NCL delay element

```
process(clk) begin
  if rising_edge(clk) then
    delay_reg <= delay_reg(d-2 downto 0) & input;
```

```

    end if;
end process;
output <= delay_reg(d-1);

```

4.3 Cell Reference Syntax

Listing 7: Cell coupling syntax

```

Monitor[($cell[42])(alert<>)
  alert<$cell[42]$threshold(>
|
  $cell[42]$threshold() : 0:0 1:0 2:0 3:0 4:0 5:1 6:1 7:1
]

Inject[($signal)(cell[44]<>)
  cell[44]<$signal>
]

```

4.4 Comparison with Conventional FPGA Simulation

Property	Conventional	MatterScript
Domain geometry	Regular grid	Arbitrary mesh
Boundary handling	Staircase approximation	Exact mesh boundary
Cell adjacency	Fixed 4/6-connected grid	Mesh edge adjacency
Propagation delay	Clock-constrained	Geometry-proportional
Timing model	Global clock	Completeness-driven
Placement objective	Minimum wire length	Geometric fidelity
Domain source	Manual grid definition	CAD / LIDAR / GeoTIFF / CA
Sensor placement	Post-processing	Embedded cell references
Arithmetic	Floating point FPU	Integer lookup tables

5 Collaborative World Modeling: AI, Engineers, and Executable Physics

The preceding sections describe MatterScript as a compiler toolchain. This section describes what it enables at a larger scale: a systematic, collaborative process for building physically-faithful executable models of real-world systems, one domain at a time, grounded in hardware at each layer.

5.1 The World Model Problem

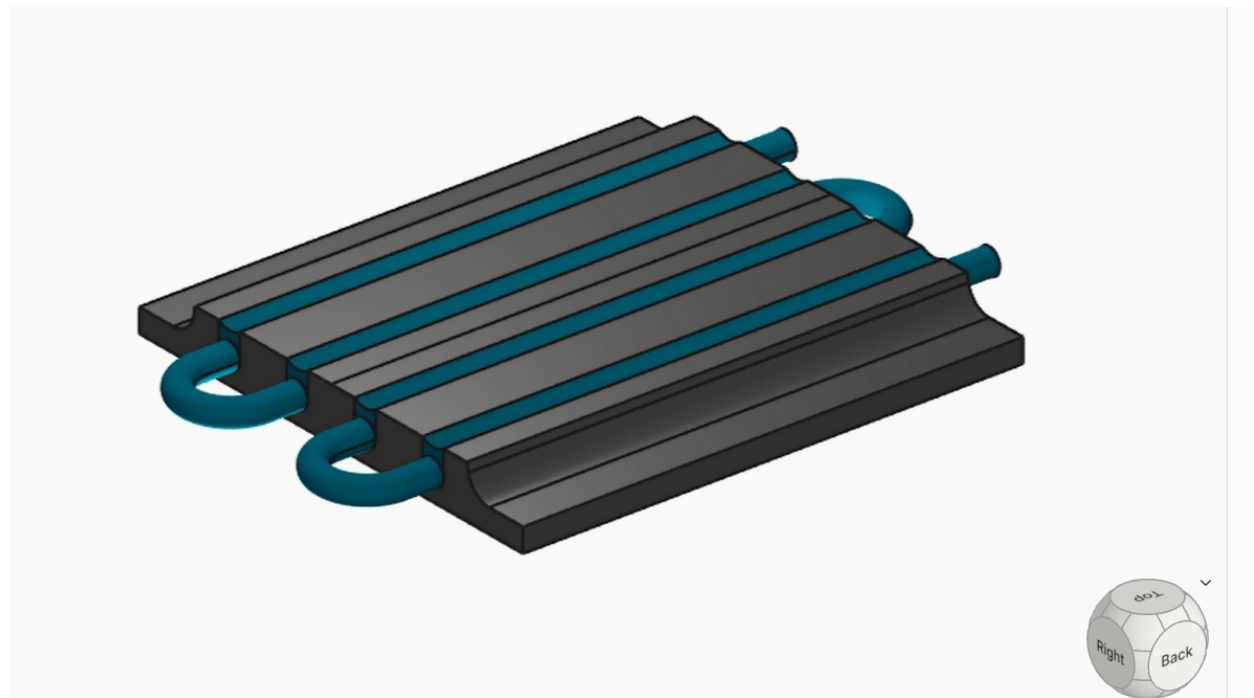
A recurring goal in artificial intelligence research is the development of *world models* — internal representations that allow reasoning systems to predict the behavior of physical

environments. The dominant approach pursues this through scale: training larger neural networks on larger datasets in the hope that a sufficiently rich model of physical reality emerges from the data.

This paper argues that this framing misidentifies where world models actually live in engineered systems. No single engineer understands a car. There are radiator engineers, motor engineers, transmission engineers, and controls engineers. The car functions not because any one person or system models all of it, but because the interfaces between subsystems are precisely specified and the behavior at each interface is verifiable. The world model of a car is distributed across its engineering documentation, its test suites, and ultimately its deployed hardware — not stored in any single representation.

MatterScript proposes a complementary approach: build world models domain by domain, collaboratively, encoding each domain’s physics into executable hardware descriptions that lock verified behavior into silicon at each layer. The interfaces between layers are then available to higher-level reasoning systems — including large language models — without those systems needing to understand the internals of the layers below.

5.2 The Radiator Example: A Case Study in Collaborative Modeling



To make this concrete, consider the problem of modeling an automotive radiator. The radiator is a heat exchanger: coolant enters through an inlet port, flows through a serpentine tube, exchanges heat with the fin array, and exits through an outlet port. The engineering problem is to model this system with sufficient fidelity to support real-time control decisions.

Step 1: Domain geometry acquisition. The radiator geometry is designed in Zoo Design Studio using parametric KCL modeling and exported as a PLY mesh. MeshLab

loads the mesh, computes per-vertex curvature and geodesic distances, and generates a UV parameterization for FPGA fabric projection.

Step 2: Vertex role assignment. The placement engine classifies mesh vertices by geometric role: inlet port vertices (open boundary, single connection), outlet port vertices, coolant channel vertices, fin surface vertices, and channel wall vertices (shared boundary between fluid and thermal domains). Classification is performed from mesh topology and curvature analysis — no manual annotation required.

Step 3: Physics definition. For each vertex role, an appropriate IL definition is selected from the MatterScript standard library:

- **FluidCell** — coolant channel vertices. Models pressure and velocity propagation: $p_{out} = \text{clamp}(p_{in} - R + v_{in}/2, 0, 7)$
- **HeatCell** — fin surface vertices. Models thermal conduction: $T_{out} = \text{clamp}((T_n + T_s + T_e + T_w)/4, 0, 7)$
- **WallCell** — channel wall vertices. Couples fluid and thermal domains at the fluid-solid boundary.
- **Inlet** — inlet port vertices. Pure source definition generating constant pressure and velocity tokens.
- **Outlet** — outlet port vertices. Absorbs tokens and emits a back-pressure token.

Step 4: Table generation. The compiler enumerates all input combinations for each definition and evaluates the physics expressions at compile time, producing VHDL case statements. No floating point arithmetic appears in the generated hardware. The serpentine coolant path acquires a natural delay structure: coolant tokens take longer to propagate from inlet to outlet than heat tokens take to conduct across a single fin, correctly reflecting physical transit times.

Step 5: Geometry-aware placement and delay synthesis. The annotated mesh is projected onto the FPGA fabric using the MeshLab UV parameterization. Synthetic delay elements are inserted on connections whose projected distance differs from their geodesic distance beyond a threshold.

Step 6: Compilation and simulation. The network compiles to VHDL, is validated by GHDL, synthesized to a Verilog netlist, and simulated by Verilator. Inlet pressure and outlet temperature are compared against physical measurements or analytical solutions.

Step 7: FPGA deployment. The validated VHDL is synthesized to an FPGA bitstream. Sensor logic at specific cell indices reads temperature and pressure at locations corresponding to physical sensor positions. Actuator logic writes boundary conditions as operating conditions change.

5.3 Lateral Scaling: From Radiator to Drivetrain

Once the radiator FPGA is deployed and its interface is defined, the next engineering problem can begin without revisiting the radiator internals. The motor controller engineer connects

to the radiator interface as a black box. The transmission engineer connects to the motor interface. At each step, the previously modeled subsystem is a deployed, verified hardware component with a precisely specified token interface.

This is not a new idea in engineering — it is how every complex system is built. What MatterScript contributes is a *formalism* for making this process explicit and executable for physics simulation workloads, and a *toolchain* that carries it from domain geometry all the way to deployed silicon without loss of physical fidelity at any layer.

The resulting system — a drivetrain modeled as a network of geometry-aware FPGA simulations, each encoding the physics of one subsystem in integer NCL logic, connected through precisely typed token interfaces — is a world model. Not a monolithic neural representation of physical reality, but a distributed, executable, hardware-verified model built by engineers who understood each piece, collaborating through interfaces that none of them had to fully understand alone.

5.4 The Role of AI in This Process

The collaborative process described in this section was itself conducted with AI assistance. The physics of heat transfer and fluid dynamics, the syntax of the Invocation Language, the compiler architecture, the FPGA placement strategy, and the component interface conventions were developed in dialogue between a human engineer with domain intuition and problem selection judgment, and an AI system with broad knowledge of physics, compiler theory, and hardware description.

Neither could have produced this system at this speed working alone. The human brought the radiator mesh, the intuition that the serpentine tube was the key physical feature, and the judgment about what level of quantization was acceptable. The AI contributed knowledge of lattice Boltzmann methods, NCL theory, VHDL generation patterns, and the observation that port vertices could be identified automatically from mesh topology.

This suggests a model for AI-assisted world modeling that differs from the superintelligence framing: not a single AI system that understands everything, but AI as a domain-knowledgeable collaborator that helps engineers make their tacit knowledge explicit and executable, one subsystem at a time. The large language model, when it eventually appears at the top of the deployed drivetrain stack, will interact with FPGA token interfaces that were built correctly because human engineers and AI systems worked together to verify each layer before moving to the next.

The world model problem may not require superintelligence. It may require exactly this: systematic, collaborative, layer-by-layer modeling of physical systems, with AI contributing knowledge and humans contributing judgment, and hardware locking each verified layer into a form that neither needs to re-derive.

6 The Compiler Pipeline

6.1 Overview

The MatterScript compiler is implemented in Zig 0.17. The current pipeline:

`.ms.il` $\xrightarrow{\text{parser}}$ Network AST $\xrightarrow{\text{emitter}}$ `.vhd` $\xrightarrow{\text{GHDL}}$ `.sv` $\xrightarrow{\text{Verilator}}$ native binary

The geometry-aware placement extension adds:

`mesh + metadata` $\xrightarrow{\text{placement}}$ annotated AST $\xrightarrow{\text{emitter}}$ `.vhd with delays` $\xrightarrow{\text{synthesis}}$ FPGA bitstream

6.2 The IL Parser

The grammar is:

```

network      = definition* entry?
definition    = name '[' dest-list source-list resolution '|' constants ']'
dest-list     = '(' ('$' name)* ')'
source-list   = '(' (name '<')* ')'
resolution    = (source-fill | invocation)*
source-fill   = name '<' expr '>'
expr          = ('$' name)+ '()'
invocation    = name '(' '(' arg* ')' source-list ')'
constants     = (composition ':' (key ':' value | generate-block)+)*
generate      = 'generate' '{' expr 'inputs:' ranges 'output:' range
               ('const' name '=' value)* '}'

```

6.3 VHDL Generation

IL concept	VHDL construct
Destination place	<code>in std_logic_vector(7 downto 0) port</code>
Source place	<code>out std_logic_vector(7 downto 0) port</code>
Completeness	Concurrent AND of all destination valid bits
Name composition	Signal concatenation of 7-bit data fields
Constant table	Combinational <code>case</code> statement
Generate block	Compile-time enumerated <code>case</code> statement
Source fill	Concurrent signal assignment
Synthetic delay	Shift register on NCL signal

6.4 Build System

- `zig build verify` — generates VHDL and runs GHDL syntax validation, cross-platform
- `zig build simulate` — full GHDL, Verilator, and testbench execution; Linux only
- `zig build test` — Zig unit tests

7 Implementation Status

Implemented and functional: FSM dialect (parser, VHDL emitter, testbench generator, full simulation pipeline); IL dialect (parser, VHDL emitter, GHDL validation); geo dialect (Rule 30 CA, mesh generator, OBJ/STL/PGM export); MKRAND deterministic random bit generator; cross-platform build system.

Designed, not yet implemented: Table generator (`generate` block); IL component instantiation; cell reference syntax and coupling table; conformal projection engine; delay synthesis; vertex role assignment; FPGA synthesis backend; simulation standard library (FluidCell, HeatCell, WallCell, Inlet, Outlet); IL testbench generator.

8 Related Work

Asynchronous logic: Fant [1] and Sparsø [7] provide the theoretical foundations. MatterScript makes delay synthesis a first-class design operation compiled from a high-level language.

FPGA-based simulation: Prior FPGA lattice Boltzmann implementations [11] use regular grid placements. MatterScript introduces geometry-faithful placement as a design objective.

Cellular automata: Wolfram [2] and Toffoli and Margolus [8] established CA as a computational model. MatterScript uses CA as a geometry generator and as the conceptual foundation for the table generator.

High-level synthesis: Chisel [10] and SpinalHDL target synchronous RTL. MatterScript targets asynchronous, geometry-aware computation.

AI world models: The limitations of purely neural world models for engineering applications [12] motivate MatterScript’s approach of making physical knowledge explicit and executable rather than implicit in model weights.

9 Conclusion

MatterScript demonstrates that geometry-aware placement of asynchronous logic networks on commodity FPGA hardware is both theoretically coherent and practically implementable. The combination of Fant’s Invocation Language, conformal projection of physical domain geometry, integer-quantized table-driven computation, and NCL delay synthesis produces simulations whose propagation behavior is faithful to the physical domain without custom hardware or floating point arithmetic.

The working compiler pipeline grounds the approach in hardware-verifiable reality. The path to full FPGA deployment is a direct extension through standard synthesis tools.

Beyond the technical contributions, MatterScript embodies a methodology for building world models the way engineers actually build complex systems: domain by domain, interface by interface, with each layer verified before the next is begun. AI systems contribute domain knowledge and pattern recognition; human engineers contribute judgment, intuition, and

problem selection. Hardware locks each verified layer into a form that neither needs to re-derive. The world model that results is not a neural approximation of physical reality but an executable, verifiable, distributed representation of it — built collaboratively, one subsystem at a time.

References

- [1] K. M. Fant, *Computer Science Reconsidered: The Invocation Model of Process Expression*. Wiley-Interscience, 2007.
- [2] S. Wolfram, *A New Kind of Science*. Wolfram Media, 2002.
- [3] P. Cignoni et al., “MeshLab: an Open-Source Mesh Processing Tool,” in *Proc. Eurographics Italian Chapter Conference*, 2008.
- [4] T. Gingold, *GHDL: Open Source VHDL Simulator*. <https://github.com/ghdl/ghdl>, 2025.
- [5] W. Snyder, *Verilator: The Fastest Verilog/SystemVerilog Simulator*. <https://www.veripool.org/verilator>, 2025.
- [6] S. Succi, *The Lattice Boltzmann Equation: For Fluid Dynamics and Beyond*. Oxford University Press, 2001.
- [7] J. Sparsø and S. Furber, *Principles of Asynchronous Circuit Design*. Springer, 2001.
- [8] T. Toffoli and N. Margolus, *Cellular Automata Machines*. MIT Press, 1987.
- [9] A. Adamatzky, *Physarum Machines: Computers from Slime Mould*. World Scientific, 2010.
- [10] J. Bachrach et al., “Chisel: Constructing Hardware in a Scala Embedded Language,” in *Proc. DAC*, 2012.
- [11] H. M. Waidyasooriya and M. Hariyama, “FPGA-Based Lattice Boltzmann Fluid Simulation,” *International Journal of Parallel Programming*, 2017.
- [12] Y. LeCun, “A Path Towards Autonomous Machine Intelligence,” *OpenReview preprint*, 2022.